

Ultrathin Bistable Composite Short Boom

Lab Report

Bayu Febriansyah

677240334

March, 2026

AE 498 Space Structures Laboratory

University of Illinois Urbana-Champaign

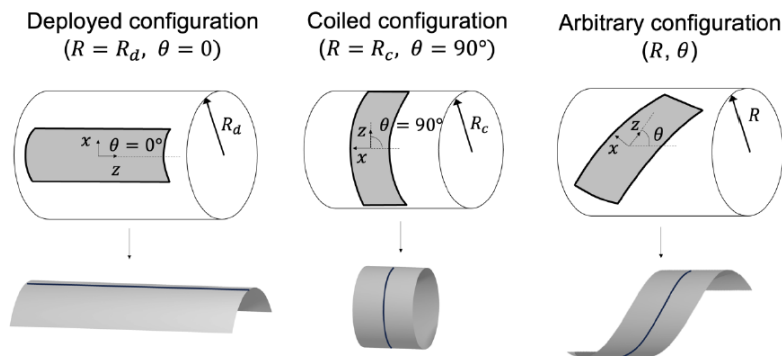
Abstract

This study investigates the bistable behavior of an ultrathin composite short boom manufactured with the layup [45PW/0UD/45PW]. The objective was to determine the analytical and experimental stable-state geometries in terms of curvature C and orientation angle θ , and to evaluate the discrepancy between theory and measured behavior. The analytical predictions were obtained using Classical Lamination Theory through the laminate A, B, and D matrices and the adjusted bending stiffness matrix, from which the boom potential energy was expressed as a function of C and θ . Stable configurations were identified as local minima on the resulting contour plot. Experimentally, the deployed and coiled boom surfaces were measured using digital image correlation and fitted to cylindrical surfaces to extract the corresponding values of C and θ . The analytical model predicted stable states at approximately $C=33.15 \text{ m}^{-1}$, $\theta=0^\circ$ for the deployed state and $C=26.35 \text{ m}^{-1}$, $\theta=90^\circ$ for the coiled state, while the experimental measurements yielded $C=31.57 \text{ m}^{-1}$, $\theta=0.46^\circ$ and $C=27.55 \text{ m}^{-1}$, $\theta=86.81^\circ$, respectively. Differences between the two were attributed to fabrication-induced deviations in effective radius, ply thickness, and ply angle, together with DIC measurement and post-processing uncertainty. A bounded parametric study showed that plausible adjustments to these quantities significantly reduced the discrepancy, while a separate design study identified a symmetric [UD/UD/UD] configuration with outer-ply angle of -25° and middle-ply angle of 30° as the configuration that minimized the estimated energy barrier between the two stable states. Overall, the results show that the analytical model captures the fundamental bistable response of the boom, while the experimental deviations highlight the sensitivity of ultrathin composite structures to real manufacturing and measurement effects.

1. Introduction

Bistable composite booms are lightweight thin-walled structures capable of maintaining two distinct equilibrium configurations without continuous external actuation. Because of their low mass, compact stowage, and passive deployment capability, they are attractive for space-structure applications. In this lab, the structure of interest is an ultrathin composite short boom fabricated from carbon-fiber prepregs in the layup [45PW/0UD/45PW] with nominal dimensions of 12 inches by 2 inches. The lab assignment requires prediction of the coiled radius from theory, fabrication of the boom, measurement of the deployed and coiled shapes using digital image correlation, and comparison between theoretical and experimental stable states. For the extended part of the work, the contour-plot framework is also used to search for a geometry that minimizes the separation or energy barrier between stable states.

The theoretical foundation of the analysis is Classical Lamination Theory. In this framework, the laminate response is described by the ABD stiffness matrix, which relates in-plane strains and curvatures to membrane forces and bending moments. The A matrix represents extensional stiffness, the B matrix captures bending-stretching coupling, and the D matrix represents bending stiffness. For a given laminate, these matrices are assembled from the transformed ply stiffnesses and ply positions through the thickness. In the present problem, this formulation is used to build the adjusted bending stiffness matrix, which governs the bending-dominated deformation of the boom.



The boom geometry is described by two parameters: the cylindrical curvature C and the angle θ between the cylinder axis and the boom longitudinal direction. In the deployed state, the boom lies on a cylinder with $\theta=0^\circ$, whereas in the coiled state the boom wraps circumferentially with $\theta=90^\circ$. Intermediate helical configurations can also exist between these two limits. Under the assumptions used in the analytical model (perfect ply bonding, small strains, thin-shell behavior, negligible membrane energy during deployment, no residual manufacturing strains,

and uniform deformation), the boom can be treated as lying on a cylinder of constant curvature throughout the motion. This allows the strain energy to be written in terms of the curvature change vector and an adjusted bending stiffness matrix, so that stable states correspond to local minima of the potential energy in (C, θ) space.

The purpose of this report is therefore fourfold:

- to determine the analytical stable-state values of C and θ using the laminate stiffness formulation and the boom potential-energy model
- to determine the real values of C and θ from DIC-based cylinder fitting of the manufactured boom
- to explain the discrepancy between the analytical and experimental results
- to identify an optimized boom geometry that minimizes the separation between stable states on the potential-energy contour plot.

Any disagreement between theory and experiment is expected because the analytical model assumes ideal material behavior and perfect geometry, while the real specimen may contain cure-induced residual stress, imperfect ply placement, thickness variation, slight geometric asymmetry, measurement noise, and fitting error. Even so, the comparison is valuable because it shows both the predictive capability and the limitations of ideal laminate theory when applied to real ultrathin bistable composite structures.

2. Analytical Model

The analytical prediction of the bistable short boom begins with Classical Lamination Theory, as required in the lab assignment for the [45PW/0UD/45PW] laminate. In this framework, the laminate stiffness is written in terms of the 6×6 ABD matrix, which relates the in-plane force resultants and bending moment resultants to the mid-plane strains and curvatures. The A matrix represents the extensional stiffness, the B matrix represents bending-extension coupling, and the D matrix represents bending and torsional stiffness.

$$\widetilde{Q}_k^r = \begin{bmatrix} \frac{E_{11}^2}{E_{11}-\nu_{12}^2 E_{22}} & \frac{\nu_{12} E_{11} E_{22}}{E_{11}-\nu_{12}^2 E_{22}} & 0 \\ \frac{\nu_{12} E_{11} E_{22}}{E_{11}-\nu_{12}^2 E_{22}} & \frac{E_{11} E_{22}}{E_{11}-\nu_{12}^2 E_{22}} & 0 \\ 0 & 0 & G_{12} \end{bmatrix}$$

$$\widetilde{Q}_k = \begin{bmatrix} Q_{11} & Q_{12} & Q_{16} \\ Q_{12} & Q_{22} & Q_{26} \\ Q_{16} & Q_{26} & Q_{66} \end{bmatrix}$$

$$Q_{11} = Q_{11}^r \cos^4 \theta + 2(Q_{12}^r + 2Q_{66}^r) \cos^2 \theta \sin^2 \theta + Q_{22}^r \sin^4 \theta$$

$$Q_{12} = Q_{12}^r (\cos^4 \theta + \sin^4 \theta) + (Q_{11}^r + Q_{22}^r - 4Q_{66}^r) \cos^2 \theta \sin^2 \theta$$

$$Q_{16} = (Q_{11}^r - Q_{12}^r - 2Q_{66}^r) \cos^3 \theta \sin \theta - (Q_{22}^r - Q_{12}^r - 2Q_{66}^r) \cos \theta \sin^3 \theta$$

$$Q_{22} = Q_{11}^r \sin^4 \theta + 2(Q_{12}^r + 2Q_{66}^r) \cos^2 \theta \sin^2 \theta + Q_{22}^r \cos^4 \theta$$

$$Q_{26} = (Q_{11}^r - Q_{12}^r - 2Q_{66}^r) \cos \theta \sin^3 \theta - (Q_{22}^r - Q_{12}^r - 2Q_{66}^r) \cos^3 \theta \sin \theta$$

$$Q_{66} = (Q_{11}^r + Q_{22}^r - 2Q_{12}^r - 2Q_{66}^r) \cos^2 \theta \sin^2 \theta + Q_{66}^r (\cos^4 \theta + \sin^4 \theta)$$

The assignment first defines the reduced ply stiffness matrix Q from the lamina elastic constants E11, E22, ν12, and G12, then transforms it according to ply angle θ, and finally assembles the laminate stiffness matrices by integrating the transformed ply stiffnesses through the thickness using the ply interface coordinates z_k. In the assignment notation, these are given by the equations below.

$$A_{ij} = \sum_{k=1}^n \{Q_{ij}\}_k (z_k - z_{k-1})$$

$$B_{ij} = \frac{1}{2} \sum_{k=1}^n \{Q_{ij}\}_k (z_k^2 - z_{k-1}^2)$$

$$D_{ij} = \frac{1}{3} \sum_{k=1}^n \{Q_{ij}\}_k (z_k^3 - z_{k-1}^3)$$

Along with the material properties below.

Property	UD prepreg	PW prepreg
E11 (GPa)	144,1	89,0
E22 (GPa)	5,2	89,0
nu12	0,335	0,035
G12 (GPa)	2,8	4,2
Ply Angle (degrees)	0	45
Thickness (μm)	40,0	58,2

The boom model used in the assignment is based on the cylindrical slit-boom formulation of Guest and Pellegrino using the following assumptions:

- perfect bonding
- small strains
- linearly elastic orthotropic plies
- thin-shell kinematics
- no residual manufacturing strain
- negligible extensional stress during deployment

Under these stated assumptions, the boom is assumed to deform at approximately constant Gaussian curvature. As a result, any admissible configuration can be described by only two geometric parameters: the curvature C of the cylinder on which the boom lies, and the angle T between the cylinder axis and the boom longitudinal direction. The deployed state corresponds to $T=0^\circ$, while the coiled state corresponds to $T=90^\circ$. The change in curvature is written in the assignment as follows along with the corresponding strain-energy density equation.

$$\tilde{k} = \begin{bmatrix} \Delta\kappa_x \\ \Delta\kappa_y \\ \Delta\kappa_{xy} \end{bmatrix} = \frac{C}{2} \begin{bmatrix} 1 - \cos(2\theta) \\ \cos(2\theta) + 1 - \frac{2}{CR_d} \\ 2 \sin(2\theta) \end{bmatrix}$$

$$U = \frac{1}{2} \tilde{k}^T \widetilde{D}^* \tilde{k}$$

$$\widetilde{D}^* = \widetilde{D} - \widetilde{B}^{-1} \widetilde{A} \widetilde{B}$$

Therefore, the stable states are identified as the local minima of $U(C, T)$.

The MATLAB code follows this analytical procedure directly. The first section defines the material properties of the UD and PW prepregs and computes each transformed ply stiffness matrix Q from the elastic constants and fiber orientation. Then it defines the ply thicknesses and interface coordinates z_0 through z_3 for the [45PW/0UD/45PW] stack, which is assembled into the laminate A , B , and D matrices. Finally, it computes the adjusted bending stiffness matrix.

Section 2 of the code sets the deployed boom radius using the mold diameter of 60.325 mm, so that R_d is equal to 0.0301625 m, and evaluates the energy over a grid of curvature values C and angular values T . These values are mapped into the contour-plot coordinates $X = C \cos(2T)$ and $Y = C \sin(2T)$, which reproduces the polar-style potential-energy plot described in the handout. Finally, the code searches the computed energy surface numerically for local minima, treating the T -direction as periodic because $T=0^\circ$ and $T=180^\circ$ represent the same physical configuration. The two lowest distinct minima are then labeled as the deployed and coiled stable states.

Using the current material inputs and geometry in the MATLAB model, the contour-search procedure predicts two analytical stable states. The first is the deployed state, located at approximately $T=0^\circ$ with curvature $C \approx 33.17 \text{ m}^{-1}$. The second is the coiled state, located at approximately $T=90^\circ$ with curvature $C \approx 26.37 \text{ m}^{-1}$. These are the numerical values returned by the present grid-based implementation; with finer resolution, they converge very closely to $C \approx 33.15 \text{ m}^{-1}$ for the deployed state and $C \approx 26.35 \text{ m}^{-1}$ for the coiled state. These two minima form the analytical reference states against which the experimental DIC-based cylinder fits can be compared.

Stable State		Analytical	Numerical
Deployed	$C \text{ (m}^{-1}\text{)}$	33.15	33.17
	$T \text{ (degrees)}$	0	0
Coiled	$C \text{ (m}^{-1}\text{)}$	26.35	26.37
	$T \text{ (degrees)}$	90	90

3. Experimental Data

The specimen was fabricated in accordance with the lab assignment for the ultrathin bistable short boom, which specifies a [45PW/0UD/45PW] laminate with overall dimensions of 2' inches by 12 inches. The prepregs were plain weave (PW) and unidirectional (UD), and the boom was formed on the cylindrical mold provided for the lab. The assignment identifies the mold diameter as 60.325 mm and lists the nominal ply thicknesses as 58.2 μm for PW and 40.0 μm for UD.



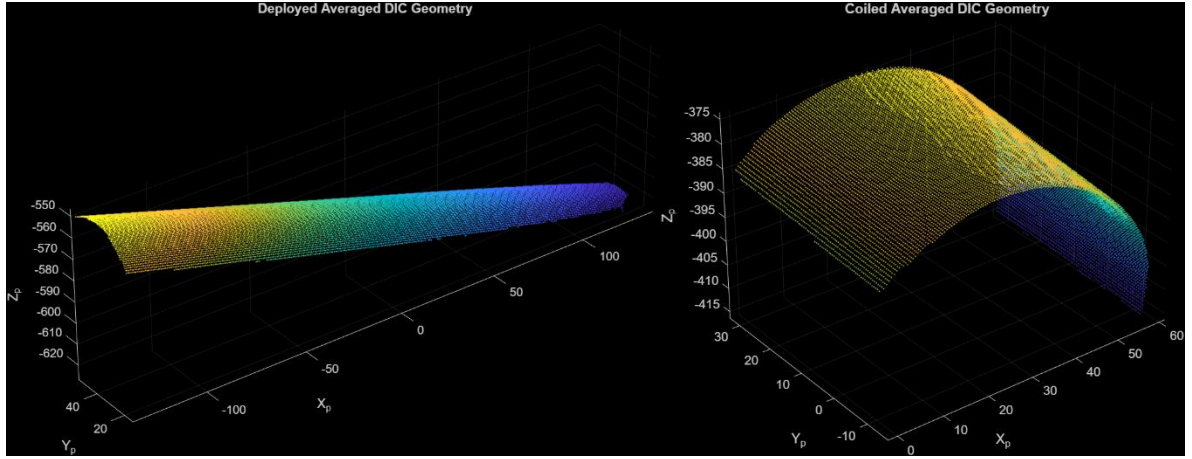
To manufacture the boom, the plies were laid up in the required order of PW, UD, and PW, with the fiber orientations arranged according to the layup diagram in the assignment. After layup, the laminate was vacuum bagged using sealing tape to close the bag around the mold and laminate stack. Vacuum was maintained through a hose connected from the bag to a vacuum pump, and the specimen was then cured in an oven while the vacuum remained applied.



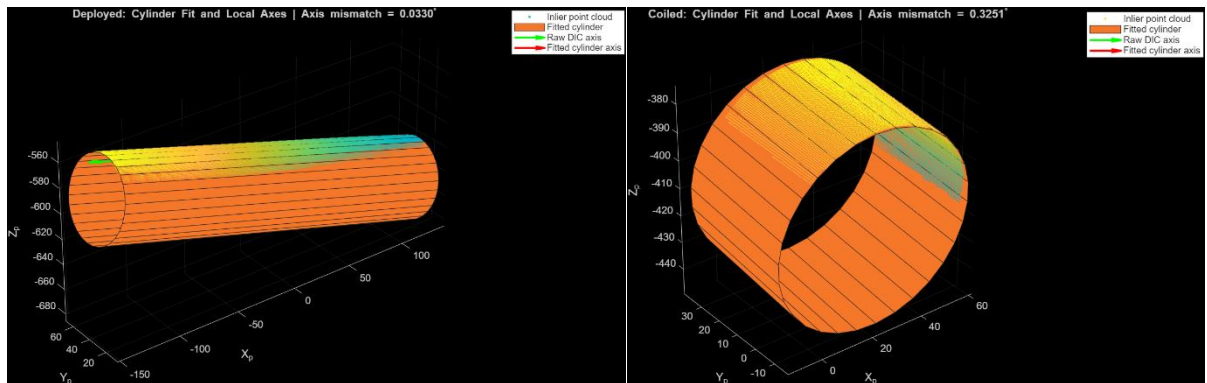
After fabrication, the two stable states of the boom were characterized experimentally using digital image correlation (DIC), which is the data-acquisition method specified in the assignment for the deployed and coiled surfaces. To enable optical tracking of the surface geometry, the outer curved face of the boom was spray painted with a speckle pattern of white dots. A two-camera DIC system with a lighting arrangement was then used to capture the boom shape. Before data collection, the camera focus, polarizer settings, and lighting configuration were adjusted to maximize captured surface area and image clarity, and the full setup was calibrated using a reference plate. Once the system was calibrated, five separate captures were taken for each stable states in order to reduce random measurement noise. The measured geometries from the five captures were then averaged pointwise to produce one representative deployed dataset and one representative coiled dataset for subsequent analysis.



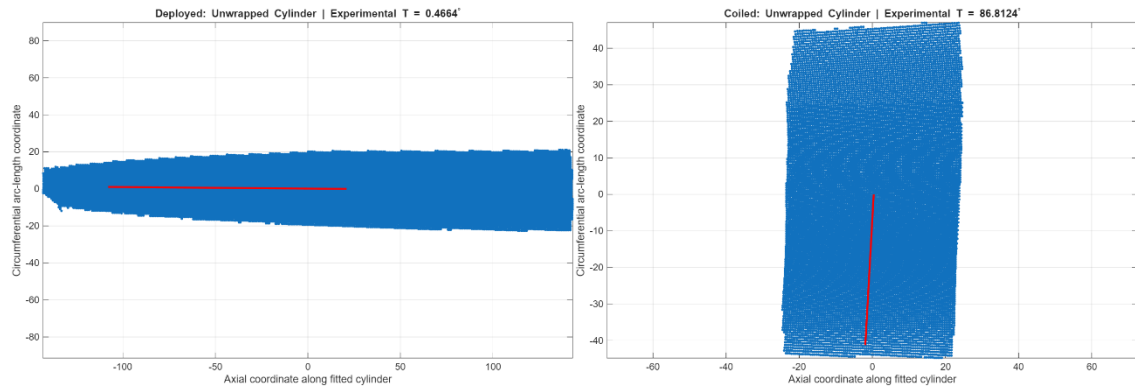
The averaged DIC data were exported to CSV format and processed in MATLAB to extract the experimental stable-state curvature C and angle T . In the analytical model used for this project, the boom geometry is described by two parameters: the cylinder curvature C and the angle θ between the cylinder axis and the boom longitudinal direction. The assignment defines the deployed state as $\theta=0^\circ$ and the coiled state as $\theta=90^\circ$, so the experimental post-processing was designed to recover these same quantities from the measured geometry.



For each CSV file, the MATLAB script first read the three spatial coordinates (x_p , y_p , z_p) and removed rows containing invalid values. The remaining points were used to generate a point-cloud representation of the measured boom surface. A cylinder was then fitted to the cloud using MATLAB's `pcfitcylinder` routine. For the deployed configuration, the cylinder was fitted without a prescribed axis direction, while for the coiled configuration an initial axis estimate from the raw surface normals was used as a reference to improve fitting robustness. The fitted cylinder provided the experimental radius, from which the experimental curvature was obtained. The scripts also reported the number of inliers and the mean fitting error, which were used as indicators of fit quality.



To recover the experimental T in a form consistent with the analytical model, the fitted cylinder was next unwrapped into a two-dimensional plane. The inlier points were expressed in a local coordinate system built from the fitted cylinder axis and two orthogonal transverse basis vectors. Each point was then mapped into an axial coordinate along the cylinder axis and a circumferential arc-length coordinate around the cylinder. In this unwrapped plane, the dominant direction of the boom geometry was found using singular value decomposition. The experimental stable-state angle was then defined as the angle between this dominant boom direction and the cylinder axis.



Because MATLAB’s `pcfitcylinder` uses the MSAC algorithm, which is a RANSAC-type randomized fitting method, repeated runs can return slightly different fitted cylinders and therefore slightly different experimental values of C and T . For that reason, the specific run selected for reporting was chosen based on the lowest mean cylinder fitting error among the candidate runs, since this corresponds to the cylinder model that best matched the inlier points in the measured DIC geometry.

Using this procedure, the MATLAB processing produced one experimental pair (C , T) for the deployed state and one for the coiled state. For the deployed configuration, the experimental curvature and angle were found to be 31.57 m^{-1} and 0.46° respectively with a mean cylinder fitting error of 0.14 mm , while for the coiled configuration, the experimental curvature and angle were found to be 27.55 m^{-1} and 86.81° with a mean cylinder fitting error of 0.11 mm .

Stable State		Experimental	Fitting Error (mm)
Deployed	C (m^{-1})	31.57	0.14
	T (degrees)	0.46	
Coiled	C (m^{-1})	27.55	0.11
	T (degrees)	86.81	

4. Discrepancy Analysis

The initial analytical model successfully captured the overall two-stable-state behavior, but noticeable discrepancies remained between the analytical and experimental values. These results indicate that the original analytical model reproduced the qualitative response correctly, but it did not fully capture the exact measured geometry in either stable state.

Stable State		Analytical	Experimental	Error (absolute)
Deployed	C (m-1)	33.15	31.57	1.58
	T (degrees)	0	0.46	0.46
Coiled	C (m-1)	26.35	27.55	1.20
	T (degrees)	90	86.81	3.19

To explain the origin of these discrepancies, a bounded parameter-adjustment study was performed using the analytical code. The purpose of this step was not to force an arbitrary fit, but to test whether reasonable manufacturing deviations could account for the difference between the ideal analytical prediction and the measured specimen. Five input parameters were selected for adjustment: the reference radius (Rd), the plain-weave ply thickness, the unidirectional ply thickness, the plain-weave ply angle, and the unidirectional ply angle. These quantities were chosen because they are the most likely to deviate during fabrication. The effective radius can increase due to the stacked vacuum bagging materials between the laminate and tooling, the ply thicknesses can change during handling and consolidation, and the ply angles can shift slightly because the lay-up was performed by hand.

Parameter (units)	Nominal	Lower Bound	Upper Bound	Step
Rd	30,1625	30,1625	40,1625	1
tPW	58,2	29,1	87,3	5
tUD	40	20	60	5
thPW	45	40	50	1
thUD	0	-5	5	1

The search boundaries were chosen to reflect the manufacturing process and to keep the adjustment physically realistic. The reference radius was allowed to increase from its nominal value, but not decrease, because the steel mold is rigid and cannot become smaller, whereas bagging materials can make the effective forming radius larger. The thicknesses were varied from half to one and a half times their nominal values to account for handling, consolidation,

and effective through-thickness variation. The ply angles were constrained within $\pm 5^\circ$ of their nominal values, since larger deviations would be unlikely for manual lay-up of this specimen.

A new MATLAB code is created based on the original analytical algorithm and adapted to perform an exhaustive search over all possible parameter combinations within these bounds. For each candidate set of input parameters, the transformed reduced stiffness matrices were recalculated, followed by the laminate A, B, and D matrices and the adjusted bending stiffness matrix DAdj. Using these updated stiffness quantities, the strain energy surface $U(T, C)$ was evaluated over a grid of curvature C and angle T . Local minima of this energy surface were interpreted as candidate stable states. The code then identified which minima corresponded to the deployed and coiled states by comparing them with the experimental values. A normalized objective function was used to rank each parameter set, so the best solution was the one that produced the smallest total discrepancy between the analytical and experimental stable states within the prescribed bounds. Because the search evaluated all parameter combinations inside the chosen limits, the final result represents the best fit available within those physically justified boundaries.

Stable State		Analytical	Experimental	Absolute Error
Coiled	C (m-1)	26,6625	27,55	0,8875
	T (degrees)	87,5	86,81	0,69
Deployed	C (m-1)	32,0625	31,57	0,4925
	T (degrees)	0	0,46	0,46

Overall, the parameter-adjusted analytical model reduced the error in both curvatures and in the coiled angle, which supports the conclusion that a significant portion of the discrepancy can be explained by realistic fabrication-induced deviations. The fitted parameter changes are also physically interpretable. The increase in effective radius is consistent with the presence of vacuum bagging layers between the laminate and the rigid tool. The 1° change in plain-weave angle and the -2° change in the unidirectional angle are also realistic for hand lay-up. The thickness changes should be interpreted more carefully. While handling and consolidation can certainly alter effective thickness, the fitted thicknesses are better understood as representing effective laminate stiffness variation rather than literal ply-thickness change alone. In practice, such effective variation can arise from resin distribution, local compaction differences, fiber-volume-fraction variation, or slight nonuniformity through the laminate.

The remaining discrepancies can reasonably be attributed to DIC measurement uncertainty and post-processing error. The DIC process introduces uncertainty through stereo calibration accuracy, camera alignment, speckle quality, lighting conditions, and subset correlation. Even after averaging multiple captures, there is still some uncertainty in the reconstructed three-dimensional geometry. Additional error is introduced during post-processing when the point-cloud data are reduced to a single curvature and angle. The extracted C and T values depend on choices such as spatial filtering, point selection, coordinate alignment, and the fitting procedure used to represent the measured geometry. Because these steps compress a complex measured shape into a few scalar quantities, small differences in fitting or alignment can produce non-negligible changes in the reported curvature and angle.

The fact that the deployed angle remained fixed at 0° is also understandable. The experimental deployed angle is only 0.46° , which is already very small. In the analytical model, the symmetric PW/UD/PW laminate configuration strongly favors a deployed minimum at exactly 0° . Because the top and bottom PW plies were constrained to have the same angle in the fitting procedure, the model preserved this symmetry, making it difficult for the deployed equilibrium to shift away from zero. In other words, the remaining 0.46° discrepancy is likely associated with slight physical asymmetry, measurement noise, or post-processing bias rather than with the dominant structural behavior captured by the model.

In summary, the deviation between the original analytical prediction and the experimental data is consistent with a combination of fabrication-induced variation and experimental uncertainty. The bounded parameter search showed that plausible changes in effective radius, ply angle, and effective thickness/stiffness can reduce the deployed curvature error from 1.58 m^{-1} to 0.4925 m^{-1} , the coiled curvature error from 1.20 m^{-1} to 0.8875 m^{-1} , and the coiled angle error from 3.19° to 0.69° . The remaining discrepancy, especially the unchanged deployed angle error of 0.46° , is most reasonably explained by DIC measurement uncertainty, geometric post-processing, and modeling assumptions such as laminate symmetry and idealized constitutive behavior.

5. Optimization Approach

The objective of this section was to identify a three-ply symmetric laminate configuration that minimizes the energy barrier between the two stable states while keeping the laminate architecture close to the original design. Rather than altering the number of plies or introducing a substantially different laminate thickness, the optimization was restricted to variations in layup sequence and ply angles only. This kept the study focused on design changes that could plausibly improve bistable behavior without significantly changing the boom geometry or mass.

The starting point for the optimization was the original three-ply concept, which consisted of a symmetric outer-middle-outer architecture. To preserve that same general structural character, the layup was constrained to remain symmetric and to contain only three plies. This decision was made for three reasons. First, increasing the number of plies would immediately change the total thickness, stiffness distribution, and weight more substantially, making it difficult to isolate the effect of laminate sequence and orientation. Second, maintaining a three-ply design keeps the optimized configuration directly comparable to the baseline laminate used in the analytical and experimental study. Finally, keeping the same prepregs also ensure that any optimized configuration that the model obtains is something that can be manufactured using the same resources available to fabricate the initial specimen. Within that three-ply symmetric constraint, four laminate families were considered:

- UD / UD / UD
- UD / PW / UD
- PW / UD / PW
- PW / PW / PW

These combinations span the possible symmetric three-ply arrangements using the two available material systems, unidirectional (UD) and plain weave (PW), while avoiding asymmetric stacking.

The optimization variables were the outer-ply angle and the middle-ply angle. These angles were varied independently over a wide range using coarse increments of 5°. Large angle increments were intentionally used at this stage because the purpose of this section was broad configuration screening rather than fine local tuning. A coarser search makes it possible to

quickly identify major design trends and promising laminate families before any subsequent refinement is attempted.

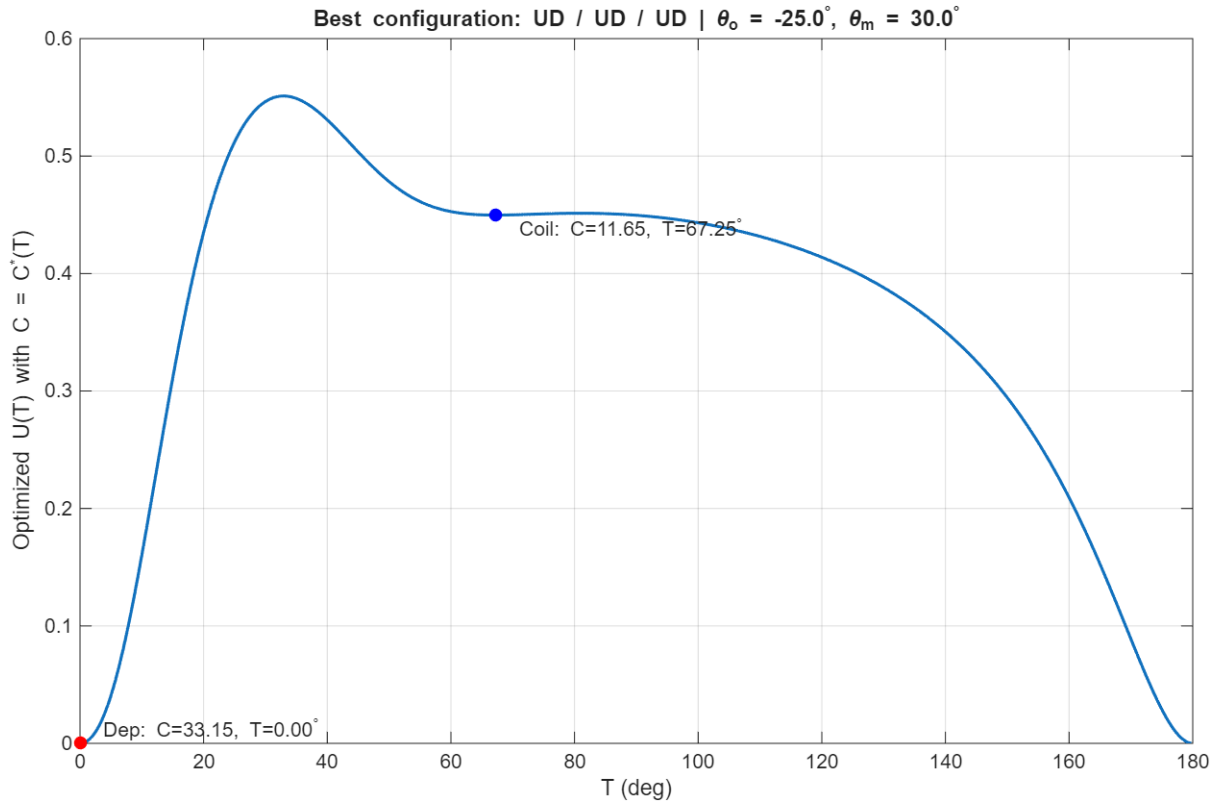
The boom radius R_d and the nominal ply thicknesses were held fixed during this study. This was a deliberate modeling choice. In the discrepancy-analysis section, radius and thickness variation were introduced to explain differences between the original analytical model and the fabricated specimen. In contrast, the purpose of the present section is not to explain fabrication error but to compare candidate laminate designs on a consistent geometric basis. Fixing the radius and nominal ply thicknesses therefore isolates the influence of layup sequence and ply orientation on bistable response.

The design objective was to minimize the energy barrier between the two stable states. Physically, the energy barrier represents the amount of strain energy that must be overcome for the structure to transition from one stable configuration to the other. A smaller barrier corresponds to an easier transition, which is desirable when seeking a configuration that can snap between states more readily.

To evaluate this, the code first computed the laminate stiffness matrices for each candidate layup-angle combination. For every ply, the transformed reduced stiffness matrix Q was generated from the specified ply angle and material properties. These transformed ply stiffnesses were then assembled into the laminate A , B , and D matrices. From these, the adjusted bending stiffness matrix D_{Adj} was constructed and used to evaluate the strain-energy landscape. The code then identified local minima corresponding to stable states. Once the two states were identified, the transition barrier was estimated by tracing a straight-line path in (C, T) space between the two minima and recording the maximum energy along that path. Three related barrier measures were reported:

- barrier from the deployed state
- barrier from the coiled state
- a common barrier measured relative to the higher-energy of the two states

The optimization criterion was the common barrier, since this provides a single measure of how easily the laminate can transition between its two stable equilibria.



The sweep examined 5476 design combinations. The optimal configuration identified by the algorithm is [UD / UD / UD] with the outer ply angle set at -25° and the inner ply angle set at 30° . However, the result also shows that minimizing the barrier alone does not preserve the original coiled geometry. The coiled-like state of the optimal design became much less curved and shifted away from 90° , demonstrating that barrier reduction and geometric retention are competing objectives. Therefore, the present optimization should be interpreted as a proof of design sensitivity and a first-stage screening study rather than a final configuration selection. It successfully identifies the layup family and angle pattern that most effectively lowers the energy barrier, but any final design recommendation should incorporate additional constraints on the stable-state geometry if maintaining the original deployed and coiled shapes is important.

6. Conclusion

This study investigated the bistable behavior of an ultrathin composite short boom with the laminate [45PW/0UD/45PW] through both analytical modeling and experimental measurement. Using Classical Lamination Theory and the cylindrical slit-boom formulation, the analytical model predicted two stable states for the nominal boom: a deployed state at approximately $C=33.15 \text{ m}^{-1}$, $T=0^\circ$, and a coiled state at approximately $C=26.35 \text{ m}^{-1}$, $T=90^\circ$. Experimentally, digital image correlation and cylinder fitting produced stable-state values of $C=31.57 \text{ m}^{-1}$, $T=0.46^\circ$ for the deployed configuration and $C=27.55 \text{ m}^{-1}$, $T=86.81^\circ$ for the coiled configuration. The comparison showed that the analytical model correctly captured the existence and qualitative nature of the two stable states, but did not exactly reproduce the measured geometry. In the initial comparison, the absolute errors were 1.58 m^{-1} and 0.46° for the deployed state, and 1.20 m^{-1} and 3.19° for the coiled state.

To explain these discrepancies, a bounded parameter-adjustment study was carried out in which the effective mold radius, plain-weave thickness, unidirectional thickness, plain-weave angle, and unidirectional angle were varied within physically justified bounds. These parameters were chosen because they are the most likely to deviate during fabrication: the effective radius can increase because of vacuum bagging layers, ply thicknesses can vary due to handling and consolidation, and ply angles can shift because the laminate was laid up by hand. The exhaustive search showed that realistic changes in these quantities can significantly reduce the mismatch between theory and experiment. The best-fit parameter set within the prescribed search region was $R_d=0.031162 \text{ m}$, $t_{PW}=74.1 \text{ }\mu\text{m}$, $t_{UD}=20.0 \text{ }\mu\text{m}$, $\theta_{PW}=46^\circ$, and $\theta_{UD}=-2^\circ$. With these adjusted inputs, the deployed curvature error was reduced from 1.58 m^{-1} to 0.4925 m^{-1} , the coiled curvature error from 1.20 m^{-1} to 0.8875 m^{-1} , and the coiled angle error from 3.19° to 0.69° , while the deployed angle remained at 0° and therefore retained an error of 0.46° . These results shows that a substantial portion of the discrepancy between analytical and experimental behavior can be explained by realistic fabrication-induced deviations.

The remaining disagreement is reasonably attributed to measurement and processing uncertainty as well as modeling assumptions. On the experimental side, the DIC results depend on stereo calibration quality, lighting, speckle pattern quality, subset tracking, point-cloud averaging, cylinder fitting, and the subsequent extraction of C and T from the fitted geometry. Small changes in any of these steps can produce noticeable changes in the final reported curvature and angle, especially for a very thin, compliant structure. On the analytical side, the

model assumes perfect bonding, thin-shell behavior, linear elasticity, no residual manufacturing strain, negligible extensional stress during deployment, and geometrically uniform cylindrical deformation. In reality, the boom may exhibit slight asymmetry, local nonuniformity, residual cure effects, or effective stiffness changes that are not fully captured by the idealized formulation. The fact that the deployed angle remained fixed at 0° even after parameter adjustment is consistent with this interpretation: the experimental deployed angle of 0.46° is very small, and the symmetric analytical model naturally favors a deployed minimum at exactly 0° , so this remaining offset is more plausibly associated with slight physical asymmetry or measurement/post-processing bias than with the dominant bistable response itself.

A further optimization study was then performed to explore how layup sequence and ply angle could be used to reduce the transition barrier between stable states while keeping the laminate symmetric and limited to three plies. Among the four three-ply symmetric layup families considered, the configuration that minimized the common barrier was [UD/UD/UD] with outer-ply angle -25° and middle-ply angle 30° . This result showed that laminate architecture and orientation have a strong influence on the bistable energy landscape. However, the optimization also showed that minimizing the barrier alone does not preserve the original coiled geometry, since the optimized coiled-like state became substantially less curved and shifted away from 90° . Therefore, this part of the study should be interpreted as a design-sensitivity and first-stage screening exercise rather than as a final boom design recommendation. It demonstrates that the stable-state energy landscape can be tailored significantly through laminate design, but also that barrier minimization and geometric retention are competing objectives that would need to be balanced in a more constrained optimization.

Overall, the report shows that Classical Lamination Theory provides a useful first-order prediction of bistable boom behavior and successfully captures the existence of deployed and coiled stable states. At the same time, the comparison with DIC measurements demonstrates that real ultrathin composite structures are highly sensitive to fabrication effects, effective stiffness variation, and experimental uncertainty. The discrepancy-analysis study showed that plausible manufacturing deviations can account for much of the difference between theory and experiment, while the optimization study showed that laminate sequence and ply orientation can be used to tune the bistable response. Taken together, these results confirm both the predictive value and the limitations of ideal laminate theory when applied to real ultrathin bistable composite booms.

Appendix

% Contour Plot Section 1

```
E11_list = [144.1e9, 89e9];  
E22_list = [5.2e9, 89e9];  
nu12_list = [0.335, 0.035];  
G12_list = [2.8e9, 4.2e9];  
th_list = [0, 45];
```

```
Q_matrices = cell(length(E11_list), 1);
```

```
for i = 1:length(E11_list)
```

```
    E11 = E11_list(i);  
    E22 = E22_list(i);  
    nu12 = nu12_list(i);  
    G12 = G12_list(i);  
    th = th_list(i);
```

```
    den = E11 - nu12^2 * E22;  
    Qr11 = E11^2 / den;  
    Qr12 = (nu12 * E11 * E22) / den;  
    Qr22 = (E11 * E22) / den;  
    Qr66 = G12;
```

```
    Q11 = Qr11*cosd(th)^4 + 2*(Qr12 + 2*Qr66)*cosd(th)^2*sind(th)^2 +  
    Qr22*sind(th)^4;
```

```
    Q12 = Qr12*(cosd(th)^4 + sind(th)^4) + (Qr11 + Qr22 -  
    4*Qr66)*cosd(th)^2*sind(th)^2;
```

```
    Q16 = (Qr11 - Qr12 - 2*Qr66)*cosd(th)^3*sind(th) - (Qr22 - Qr12 -  
    2*Qr66)*cosd(th)*sind(th)^3;
```

```
    Q22 = Qr11*sind(th)^4 + 2*(Qr12 + 2*Qr66)*cosd(th)^2*sind(th)^2 +  
    Qr22*cosd(th)^4;
```

```
    Q26 = (Qr11 - Qr12 - 2*Qr66)*cosd(th)*sind(th)^3 - (Qr22 - Qr12 -  
    2*Qr66)*cosd(th)^3*sind(th);
```

```
    Q66 = (Qr11 + Qr22 - 2*Qr12 - 2*Qr66)*cosd(th)^2*sind(th)^2 + Qr66*(cosd(th)^4  
    + sind(th)^4);
```

```
    Q_matrices{i} = [Q11, Q12, Q16;  
                     Q12, Q22, Q26;  
                     Q16, Q26, Q66];
```

```
end
```

```
QUD = Q_matrices{1};  
QPW = Q_matrices{2};
```

```
tPW = 58.2e-6;  
tUD = 40e-6;  
t = tPW + tUD + tPW;
```

```
Z0 = -t/2;  
Z1 = Z0 + tPW;  
Z2 = Z1 + tUD;  
Z3 = Z2 + tPW;
```

```
A = QPW*(Z1-Z0) + QUD*(Z2-Z1) + QPW*(Z3-Z2);  
B = 0.5*(QPW*(Z1^2-Z0^2) + QUD*(Z2^2-Z1^2) + QPW*(Z3^2-Z2^2));  
D = (1/3)*(QPW*(Z1^3-Z0^3) + QUD*(Z2^3-Z1^3) + QPW*(Z3^3-Z2^3));
```

```
DAdj = D - (B'*(A\B));
```

```
% Contour Plot Section 2
```

```
Rd = 0.5 * 0.060325;
```

```
C_vals = linspace(1e-6, 90, 2001);
```

```
T_vals = linspace(0, 180, 2001);
```

```
T_vals(end) = [];
```

```
nC = numel(C_vals);
```

```
nT = numel(T_vals);
```

```
U_vals = zeros(nC, nT);
```

```
X      = zeros(nC, nT);
```

```
Y      = zeros(nC, nT);
```

```
for i = 1:nC
```

```
    current_C = C_vals(i);
```

```
    for j = 1:nT
```

```
        current_T = T_vals(j);
```

```
        vec = (current_C/2) * [1 - cosd(2*current_T);  
                                cosd(2*current_T) + 1 - (2/(current_C*Rd));  
                                2*sind(2*current_T)];
```

```
        U_vals(i,j) = 0.5 * (vec' * DAdj * vec);
```

```
        X(i,j) = current_C * cosd(2*current_T);
```

```
        Y(i,j) = current_C * sind(2*current_T);
```

```
    end
```

```
end
```

```
figure('Color','w');
```

```
contourf(X, Y, U_vals, 50, 'LineColor', 'none');
```

```
colorbar;
```

```
xlabel('C cos(2T)');
```

```
ylabel('C sin(2T)');
```

```
title('Contour Plot of U(T,C)');
```

```
axis equal;
```

```
grid on;
```

```
hold on;
```

```
isLocalMin = false(size(U_vals));
```

```
for i = 1:nC
```

```
    for j = 1:nT
```

```
        centerVal = U_vals(i,j);
```

```
        if ~isfinite(centerVal)
```

```
            continue;
```

```
        end
```

```
        i_list = max(1, i-1):min(nC, i+1);
```

```

j_left = j - 1;
j_right = j + 1;

if j_left < 1
    j_left = nT;
end
if j_right > nT
    j_right = 1;
end

j_list = [j_left, j, j_right];

neighborhood = U_vals(i_list, j_list);
neighVals = neighborhood(:);

idxCenter = find(neighVals == centerVal, 1, 'first');
if ~isempty(idxCenter)
    neighVals(idxCenter) = [];
end

if all(centerVal <= neighVals)
    isLocalMin(i,j) = true;
end
end
end

[rowMin, colMin] = find(isLocalMin);

candC = C_vals(rowMin).';
candT = T_vals(colMin).';
candU = U_vals(sub2ind(size(U_vals), rowMin, colMin));
candX = X(sub2ind(size(X), rowMin, colMin));
candY = Y(sub2ind(size(Y), rowMin, colMin));

[candU, order] = sort(candU);
candC = candC(order);
candT = candT(order);
candX = candX(order);
candY = candY(order);

tolC = 3 * (C_vals(2) - C_vals(1));
tolT = 3 * (T_vals(2) - T_vals(1));

selC = [];
selT = [];
selU = [];
selX = [];
selY = [];

for k = 1:numel(candU)
    if isempty(selC)
        selC(end+1,1) = candC(k);
        selT(end+1,1) = candT(k);
        selU(end+1,1) = candU(k);
        selX(end+1,1) = candX(k);
        selY(end+1,1) = candY(k);
    else
        dist = sqrt(((candC(k)-selC)/tolC).^2 + ((candT(k)-selT)/tolT).^2);
        if all(dist > 1)

```

```

        selC(end+1,1) = candC(k);
        selT(end+1,1) = candT(k);
        selU(end+1,1) = candU(k);
        selX(end+1,1) = candX(k);
        selY(end+1,1) = candY(k);
    end
end
end

nStable = min(2, numel(selU));

if nStable < 2
    warning('Only %d stable state(s) detected.', nStable);
end

fprintf('\nDetected stable states:\n\n');

for k = 1:nStable
    if abs(selT(k) - 90) < 5
        stateName = 'Coiled';
    else
        stateName = 'Deployed';
    end

    fprintf('%s state:\n', stateName);
    fprintf('  C = %.6f\n', selC(k));
    fprintf('  T = %.6f deg\n', selT(k));
    fprintf('  X = %.6f\n', selX(k));
    fprintf('  Y = %.6f\n', selY(k));
    fprintf('  U = %.6e\n\n', selU(k));
end

plot(selX(1:nStable), selY(1:nStable), 'rp', ...
    'MarkerSize', 12, 'MarkerFaceColor', 'r', 'LineWidth', 1.5);

for k = 1:nStable

    if abs(selT(k) - 90) < 5
        stateName = 'Coiled';
        dx = -20;
        dy = 20;
    else
        stateName = 'Deployed';
        dx = 12;
        dy = 20;
    end

    x_text = selX(k) + dx;
    y_text = selY(k) + dy;

    plot([selX(k), x_text], [selY(k), y_text], 'w-', 'LineWidth', 1.2);

    text(x_text, y_text, ...
        sprintf('%s\nC = %.3f\nT = %.3f^\circ', stateName, selC(k), selT(k)), ...
        'Color', 'w', ...
        'FontWeight', 'bold', ...
        'FontSize', 10, ...
        'BackgroundColor', 'k', ...
        'Margin', 6, ...

```

```

        'HorizontalAlignment', 'center');
end

% Deployed DIC
clear
clc
close all

%% User settings
csvFile = 'deployedaveraged_xp_yp_zp.csv';

% Set this correctly:
% 1e-3 if DIC coordinates are in mm
% 1    if DIC coordinates are already in m
unitScaleToMeter = 1e-3;

useDenoise = true;
showFigures = true;

%% Read CSV
Ttbl = readtable(csvFile);

% Prefer named columns if they exist; otherwise use last 3 columns
varNames = lower(string(Ttbl.Properties.VariableNames));
if all(ismember(["xp", "yp", "zp"], varNames))
    xp = Ttbl{:, strcmp(varNames, "xp")};
    yp = Ttbl{:, strcmp(varNames, "yp")};
    zp = Ttbl{:, strcmp(varNames, "zp")};
else
    if width(Ttbl) < 3
        error('CSV must contain at least 3 columns.');
    end
    xyzTmp = Ttbl{:, end-2:end};
    xp = xyzTmp(:,1);
    yp = xyzTmp(:,2);
    zp = xyzTmp(:,3);
end

%% Remove invalid rows
valid = isfinite(xp) & isfinite(yp) & isfinite(zp);
xyz = [xp(valid), yp(valid), zp(valid)];

if size(xyz,1) < 50
    error('Not enough valid points for cylinder fitting.');
end

%% Optional ROI cropping
% Uncomment and adapt if needed
% xlimROI = [-inf inf];
% ylimROI = [-inf inf];
% zlimROI = [-inf inf];
% keepROI = xyz(:,1)>=xlimROI(1) & xyz(:,1)<=xlimROI(2) & ...
%           xyz(:,2)>=ylimROI(1) & xyz(:,2)<=ylimROI(2) & ...
%           xyz(:,3)>=zlimROI(1) & xyz(:,3)<=zlimROI(2);
% xyz = xyz(keepROI,:);

%% Create point cloud
ptCloud = pointCloud(xyz);

```

```

if useDenoise && exist('pcdenoise','file') == 2
    ptCloud = pcdenoise(ptCloud);
    xyz = ptCloud.Location;
end

%% Visualize raw geometry
if showFigures
    figure('Color','w')
    pcshow(ptCloud)
    xlabel('X_p')
    ylabel('Y_p')
    zlabel('Z_p')
    title('Deployed Averaged DIC Geometry')
    axis equal
    grid on
end

%% Fit cylinder (unconstrained for deployed)
bbox = max(xyz,[],1) - min(xyz,[],1);
maxDistance = 0.01 * max(bbox); % tune if needed

[model, inlierIndices, outlierIndices, meanError] = pcfitcylinder( ...
    ptCloud, maxDistance, ...
    'MaxNumTrials', 5000, ...
    'Confidence', 99.9);

if isempty(inlierIndices) || model.Radius <= 0
    error(['pcfitcylinder failed. Try changing maxDistance, ' ...
        'cropping a tighter ROI, or denoising the cloud.']);
end

%% Extract fitted cylinder properties
radius_dataUnits = model.Radius;
radius_m = radius_dataUnits * unitScaleToMeter;
C_exp = 1 / radius_m;

fitAxis = model.Orientation(:);
fitAxis = fitAxis / norm(fitAxis);

center = model.Center(:).';
height = model.Height;

inlierCloud = select(ptCloud, inlierIndices);
xyzIn = inlierCloud.Location;

inlierRatio = numel(inlierIndices) / size(xyz,1);

%% Estimate raw DIC local cylinder axis from inlier normals
k = min(30, max(10, floor(size(xyzIn,1)/50)));
rawNormals = pcnormals(inlierCloud, k);

goodN = all(isfinite(rawNormals),2) & vecnorm(rawNormals,2,2) > 0;
N = rawNormals(goodN,:);
N = N ./ vecnorm(N,2,2);

[~,~,Vn] = svd(N, 'econ');
rawAxis = Vn(:,end);
rawAxis = rawAxis / norm(rawAxis);

```

```

if dot(rawAxis, fitAxis) < 0
    rawAxis = -rawAxis;
end

axisFitMismatch_deg = acosd(min(1, max(-1, dot(rawAxis, fitAxis))));

%% Compute experimental T from unwrapped fitted cylinder
% T is the angle between the boom longitudinal direction and cylinder axis.
% We estimate it by:
% 1) expressing inlier points in fitted cylinder coordinates
% 2) unwrapping cylinder to (axial, circumferential arc-length) plane
% 3) finding dominant strip direction in that plane via SVD/PCA

% Build orthonormal basis {e1, e2, fitAxis}
helper = [1;0;0];
if abs(dot(helper, fitAxis)) > 0.9
    helper = [0;1;0];
end

e1 = helper - dot(helper, fitAxis) * fitAxis;
e1 = e1 / norm(e1);
e2 = cross(fitAxis, e1);
e2 = e2 / norm(e2);

P = xyzIn - center; % Nx3
axialCoord = P * fitAxis; % coordinate along cylinder axis

radialVec = P - axialCoord * fitAxis.'; % remove axial component
thetaRaw = atan2(radialVec * e2, radialVec * e1);

% Center angular coordinate around circular mean to avoid branch cut issues
thetaMean = angle(mean(exp(1i*thetaRaw)));
thetaCentered = angle(exp(1i*(thetaRaw - thetaMean)));

% Circumferential arc-length coordinate on unwrapped cylinder
sCoord = radius_dataUnits * thetaCentered;

% 2D unwrapped coordinates
planePts = [axialCoord, sCoord];
planeCtr = mean(planePts, 1);
planePtsCentered = planePts - planeCtr;

% Dominant direction of strip on unwrapped plane
[~,~,V2] = svd(planePtsCentered, 'econ');
boomDir2D = V2(:,1); % [component along axis, component along circumference]

% Experimental T: angle from cylinder axis to boom direction
T_exp_deg = atan2d(abs(boomDir2D(2)), abs(boomDir2D(1)));

%% Visualization: fitted cylinder and axes
if showFigures
    anchor = mean(xyzIn,1);
    scale = 0.5 * max(bbox);

    figure('Color','w')
    pcshow(inlierCloud)
    hold on
    plot(model)

```

```

quiver3(anchor(1), anchor(2), anchor(3), ...
        scale*rawAxis(1), scale*rawAxis(2), scale*rawAxis(3), ...
        0, 'g', 'LineWidth', 2);

quiver3(anchor(1), anchor(2), anchor(3), ...
        scale*fitAxis(1), scale*fitAxis(2), scale*fitAxis(3), ...
        0, 'r', 'LineWidth', 2);

xlabel('X_p')
ylabel('Y_p')
zlabel('Z_p')
title(sprintf('Deployed: Cylinder Fit and Local Axes | Axis mismatch =
%.4f^\circ', ...
            axisFitMismatch_deg))
axis equal
grid on
legend('Inlier point cloud','Fitted cylinder','Raw DIC axis','Fitted cylinder
axis')
end

%% Visualization: unwrapped cylinder and longitudinal direction
if showFigures
    figure('Color','w')
    plot(planePts(:,1), planePts(:,2), '.', 'MarkerSize', 8)
    hold on
    dirScale = 0.45 * max(max(planePts,[],1) - min(planePts,[],1));
    qx = [planeCtr(1), planeCtr(1) + dirScale*boomDir2D(1)];
    qy = [planeCtr(2), planeCtr(2) + dirScale*boomDir2D(2)];
    plot(qx, qy, 'r-', 'LineWidth', 2)

    xlabel('Axial coordinate along fitted cylinder')
    ylabel('Circumferential arc-length coordinate')
    title(sprintf('Deployed: Unwrapped Cylinder | Experimental T = %.4f^\circ',
T_exp_deg))
    axis equal
    grid on
end

%% Print results
fprintf('\n===== DEPLOYED RESULTS =====\n');
fprintf('Number of valid points                = %d\n', size(xyz,1));
fprintf('Number of inliers                      = %d\n',
numel(inlierIndices));
fprintf('Inlier ratio                            = %.4f\n', inlierRatio);
fprintf('Fitted cylinder radius (data units)       = %.6f\n', radius_dataUnits);
fprintf('Fitted cylinder radius (m)                = %.6f\n', radius_m);
fprintf('Experimental curvature C = 1/R (1/m)      = %.6f\n', C_exp);
fprintf('Mean cylinder fitting error               = %.6f\n', meanError);
fprintf('Raw DIC local axis                      = [%.6f %.6f %.6f]\n', ...
        rawAxis(1), rawAxis(2), rawAxis(3));
fprintf('Fitted cylinder axis                    = [%.6f %.6f %.6f]\n', ...
        fitAxis(1), fitAxis(2), fitAxis(3));
fprintf('Axis-fit mismatch angle (deg)             = %.6f\n',
axisFitMismatch_deg);
fprintf('Experimental stable-state angle T (deg)   = %.6f\n', T_exp_deg);
fprintf('Cylinder center                        = [%.6f %.6f %.6f]\n', ...
        center(1), center(2), center(3));
fprintf('Cylinder height                        = %.6f\n', height);

```

```

% Coiled DIC
clear
clc
close all

%% User settings
csvFile = 'coiledaveraged_xp_yp_zp.csv';

% Set this correctly:
% 1e-3 if DIC coordinates are in mm
% 1 if DIC coordinates are already in m
unitScaleToMeter = 1e-3;

useDenoise = true;
showFigures = true;

%% Read CSV
Ttbl = readtable(csvFile);

% Prefer named columns if they exist; otherwise use last 3 columns
varNames = lower(string(Ttbl.Properties.VariableNames));
if all(ismember(["xp","yp","zp"], varNames))
    xp = Ttbl{:, strcmp(varNames,"xp")};
    yp = Ttbl{:, strcmp(varNames,"yp")};
    zp = Ttbl{:, strcmp(varNames,"zp")};
else
    if width(Ttbl) < 3
        error('CSV must contain at least 3 columns.');
```

```

end

%% Visualize raw geometry
if showFigures
    figure('Color','w')
    pcshow(ptCloud)
    xlabel('X_p')
    ylabel('Y_p')
    zlabel('Z_p')
    title('Coiled Averaged DIC Geometry')
    axis equal
    grid on
end

%% Estimate raw local axis first from normals
k = min(30, max(10, floor(size(xyz,1)/50)));
rawNormals0 = pcnormals(ptCloud, k);

goodN0 = all(isfinite(rawNormals0),2) & vecnorm(rawNormals0,2,2) > 0;
N0 = rawNormals0(goodN0,:);
N0 = N0 ./ vecnorm(N0,2,2);

[~,~,V0] = svd(N0, 'econ');
rawAxis0 = V0(:,end);
rawAxis0 = rawAxis0 / norm(rawAxis0);

%% Fit cylinder (constrained using raw-axis estimate)
bbox = max(xyz,[],1) - min(xyz,[],1);
maxDistance = 0.01 * max(bbox); % tune if needed
referenceVector = rawAxis0.';
maxAngularDistance = 25; % can loosen to 30 if needed

[model, inlierIndices, outlierIndices, meanError] = pcfitcylinder( ...
    ptCloud, maxDistance, referenceVector, maxAngularDistance, ...
    'MaxNumTrials', 5000, ...
    'Confidence', 99.9);

if isempty(inlierIndices) || model.Radius <= 0
    error(['pcfitcylinder failed. Try increasing maxDistance, ' ...
        'loosening maxAngularDistance, or cropping a tighter ROI.']);
end

%% Extract fitted cylinder properties
radius_dataUnits = model.Radius;
radius_m = radius_dataUnits * unitScaleToMeter;
C_exp = 1 / radius_m;

fitAxis = model.Orientation(:);
fitAxis = fitAxis / norm(fitAxis);

center = model.Center(:).';
height = model.Height;

inlierCloud = select(ptCloud, inlierIndices);
xyzIn = inlierCloud.Location;

inlierRatio = numel(inlierIndices) / size(xyz,1);

%% Estimate raw DIC local cylinder axis from inlier normals

```

```

k2 = min(30, max(10, floor(size(xyzIn,1)/50)));
rawNormals = pcnormals(inlierCloud, k2);

goodN = all(isfinite(rawNormals),2) & vecnorm(rawNormals,2,2) > 0;
N = rawNormals(goodN,:);
N = N ./ vecnorm(N,2,2);

[~,~,Vn] = svd(N, 'econ');
rawAxis = Vn(:,end);
rawAxis = rawAxis / norm(rawAxis);

if dot(rawAxis, fitAxis) < 0
    rawAxis = -rawAxis;
end

axisFitMismatch_deg = acosd(min(1, max(-1, dot(rawAxis, fitAxis))));

%% Compute experimental T from unwrapped fitted cylinder
% T is the angle between the boom longitudinal direction and cylinder axis.

helper = [1;0;0];
if abs(dot(helper, fitAxis)) > 0.9
    helper = [0;1;0];
end

e1 = helper - dot(helper, fitAxis) * fitAxis;
e1 = e1 / norm(e1);
e2 = cross(fitAxis, e1);
e2 = e2 / norm(e2);

P = xyzIn - center;
axialCoord = P * fitAxis;

radialVec = P - axialCoord * fitAxis.';
thetaRaw = atan2(radialVec * e2, radialVec * e1);

thetaMean = angle(mean(exp(1i*thetaRaw)));
thetaCentered = angle(exp(1i*(thetaRaw - thetaMean)));

sCoord = radius_dataUnits * thetaCentered;

planePts = [axialCoord, sCoord];
planeCtr = mean(planePts, 1);
planePtsCentered = planePts - planeCtr;

[~,~,V2] = svd(planePtsCentered, 'econ');
boomDir2D = V2(:,1);

T_exp_deg = atan2d(abs(boomDir2D(2)), abs(boomDir2D(1)));

%% Visualization: fitted cylinder and axes
if showFigures
    anchor = mean(xyzIn,1);
    scale = 0.5 * max(bbox);

    figure('Color','w')
    pcshow(inlierCloud)
    hold on
    plot(model)

```

```

quiver3(anchor(1), anchor(2), anchor(3), ...
        scale*rawAxis(1), scale*rawAxis(2), scale*rawAxis(3), ...
        0, 'g', 'LineWidth', 2);

quiver3(anchor(1), anchor(2), anchor(3), ...
        scale*fitAxis(1), scale*fitAxis(2), scale*fitAxis(3), ...
        0, 'r', 'LineWidth', 2);

xlabel('X_p')
ylabel('Y_p')
zlabel('Z_p')
title(sprintf('Coiled: Cylinder Fit and Local Axes | Axis mismatch =
%.4f^\circ', ...
            axisFitMismatch_deg))
axis equal
grid on
legend('Inlier point cloud','Fitted cylinder','Raw DIC axis','Fitted cylinder
axis')
end

%% Visualization: unwrapped cylinder and longitudinal direction
if showFigures
    figure('Color','w')
    plot(planePts(:,1), planePts(:,2), '.', 'MarkerSize', 8)
    hold on
    dirScale = 0.45 * max(max(planePts,[],1) - min(planePts,[],1));
    qx = [planeCtr(1), planeCtr(1) + dirScale*boomDir2D(1)];
    qy = [planeCtr(2), planeCtr(2) + dirScale*boomDir2D(2)];
    plot(qx, qy, 'r-', 'LineWidth', 2)

    xlabel('Axial coordinate along fitted cylinder')
    ylabel('Circumferential arc-length coordinate')
    title(sprintf('Coiled: Unwrapped Cylinder | Experimental T = %.4f^\circ',
T_exp_deg))
    axis equal
    grid on
end

%% Print results
fprintf('\n==== COILED RESULTS =====\n');
fprintf('Number of valid points                = %d\n', size(xyz,1));
fprintf('Number of inliers                        = %d\n',
numel(inlierIndices));
fprintf('Inlier ratio                                = %.4f\n', inlierRatio);
fprintf('Fitted cylinder radius (data units)         = %.6f\n', radius_dataUnits);
fprintf('Fitted cylinder radius (m)                  = %.6f\n', radius_m);
fprintf('Experimental curvature C = 1/R (1/m)         = %.6f\n', C_exp);
fprintf('Mean cylinder fitting error                 = %.6f\n', meanError);
fprintf('Raw DIC local axis                          = [%.6f %.6f %.6f]\n', ...
        rawAxis(1), rawAxis(2), rawAxis(3));
fprintf('Fitted cylinder axis                       = [%.6f %.6f %.6f]\n', ...
        fitAxis(1), fitAxis(2), fitAxis(3));
fprintf('Axis-fit mismatch angle (deg)               = %.6f\n',
axisFitMismatch_deg);
fprintf('Experimental stable-state angle T (deg)     = %.6f\n', T_exp_deg);
fprintf('Cylinder center                           = [%.6f %.6f %.6f]\n', ...
        center(1), center(2), center(3));
fprintf('Cylinder height                           = %.6f\n', height);

```

```

% Discrepancy Analysis
clear
clc
close all

%% =====
% EXPERIMENTAL TARGETS
% =====
target.C_dep = 31.57; % 1/m
target.T_dep = 0.46; % deg
target.C_coil = 27.55; % 1/m
target.T_coil = 86.81; % deg

%% =====
% MATERIAL PROPERTIES
% 1 = UD
% 2 = PW
% =====
matUD.E11 = 144.1e9;
matUD.E22 = 5.2e9;
matUD.nu12 = 0.335;
matUD.G12 = 2.8e9;

matPW.E11 = 89e9;
matPW.E22 = 89e9;
matPW.nu12 = 0.035;
matPW.G12 = 4.2e9;

%% =====
% NOMINAL / REFERENCE PARAMETERS
% =====
Rd0 = 0.5 * 0.060325; % m

thPW0 = 45; % deg, same for top and bottom PW
thUD0 = 0; % deg

tPW0 = 58.2e-6; % m
tUD0 = 40e-6; % m

%% =====
% EXHAUSTIVE SEARCH SETTINGS (ADJUSTABLE)
% =====
dRd = 1e-3; % m
dAng = 1.0; % deg
dtPW = 5e-6; % m
dtUD = 5e-6; % m

nC = 801;
nT = 721; % last point removed later so 180 is not duplicated

%% =====
% PARAMETER BOUNDARIES
% =====
% Rd: only larger than nominal, up to +1 cm
Rd_vals = Rd0:dRd:(Rd0 + 0.01);

% Thicknesses: 0.5x to 1.5x nominal
tPW_vals = (0.5*tPW0):dtPW:(1.5*tPW0);
tUD_vals = (0.5*tUD0):dtUD:(1.5*tUD0);

```

```

% Angles: start at nominal, then diverge outward within +/-5 deg
thPW_vals_deg = centeredSweep(thPW0, 5, dAng);
thUD_vals_deg = centeredSweep(thUD0, 5, dAng);

%% =====
% PRINT SEARCH BOUNDARIES
% =====
fprintf('\n===== SEARCH BOUNDARIES =====\n');
fprintf('Rd          : %.6f to %.6f m, step = %.6f m, count = %d\n', ...
        min(Rd_vals), max(Rd_vals), dRd, numel(Rd_vals));

fprintf('tPW          : %.3f to %.3f um, step = %.3f um, count = %d\n', ...
        min(tPW_vals)*1e6, max(tPW_vals)*1e6, dtPW*1e6, numel(tPW_vals));

fprintf('tUD          : %.3f to %.3f um, step = %.3f um, count = %d\n', ...
        min(tUD_vals)*1e6, max(tUD_vals)*1e6, dtUD*1e6, numel(tUD_vals));

fprintf('thPW         : %.2f to %.2f deg, step = %.2f deg, count = %d\n', ...
        min(thPW_vals_deg), max(thPW_vals_deg), dAng, numel(thPW_vals_deg));

fprintf('thUD         : %.2f to %.2f deg, step = %.2f deg, count = %d\n', ...
        min(thUD_vals_deg), max(thUD_vals_deg), dAng, numel(thUD_vals_deg));

fprintf('State grid: nC = %d, nT = %d, total state points = %d\n', ...
        nC, nT-1, nC*(nT-1));

totalComb = numel(Rd_vals) * numel(tPW_vals) * numel(tUD_vals) * ...
            numel(thPW_vals_deg) * numel(thUD_vals_deg);

fprintf('Total parameter combinations to check = %.0f\n', totalComb);
fprintf('===== \n\n');

%% =====
% STATE-SPACE GRID
% =====
C_vals = linspace(1e-6, 90, nC);
T_vals = linspace(0, 180, nT);
T_vals(end) = []; % remove 180 because it is equivalent to 0

[Cg, Tg] = ndgrid(C_vals, T_vals);

%% =====
% EXHAUSTIVE SEARCH
% =====
best.obj = inf;
best.found = false;

checkedCount = 0;
progressEvery = max(1, round(totalComb/100)); % roughly every 1%

tic

for iRd = 1:numel(Rd_vals)
    Rd = Rd_vals(iRd);

    for itPW = 1:numel(tPW_vals)
        tPW = tPW_vals(itPW);

```

```

for itUD = 1:numel(tUD_vals)
    tUD = tUD_vals(itUD);

    for iPW = 1:numel(thPW_vals_deg)
        thPW = thPW_vals_deg(iPW);
        QPW = buildQbar(matPW, thPW);

        for iUDang = 1:numel(thUD_vals_deg)
            thUD = thUD_vals_deg(iUDang);
            QUD = buildQbar(matUD, thUD);

            checkedCount = checkedCount + 1;

            [A, B, D, DAdj] = buildABD(QPW, QUD, QPW, tPW, tUD);

            if rcond(A) < 1e-14 || any(~isfinite(DAdj(:)))
                continue
            end

            U_vals = energySurface(DAdj, Rd, Cg, Tg);

            minima = extractLocalMinima(U_vals, C_vals, T_vals);

            if numel(minima.C) < 2
                continue
            end

            [states, ok] = assignStableStates(minima, target);

            if ~ok
                continue
            end

            obj = ...
                states.dep.errC_abs / target.C_dep + ...
                states.dep.errT_abs / 90 + ...
                states.coil.errC_abs / target.C_coil + ...
                states.coil.errT_abs / 90;

            if obj < best.obj
                best.obj = obj;
                best.found = true;

                best.Rd = Rd;
                best.tPW = tPW;
                best.tUD = tUD;
                best.thPW = thPW;
                best.thUD = thUD;

                best.states = states;
                best.U_vals = U_vals;

            fprintf('\n=====');

            fprintf('NEW BEST FOUND\n');
            fprintf('Iteration : %d / %.0f (0.2f%% complete)\n', ...
                checkedCount, totalComb, 100*checkedCount/totalComb);
            fprintf('Objective : %.6f\n', best.obj);

```

```

        fprintf('Current best parameters:\n');
        fprintf('  Rd   = %.6f m   (%.3f mm)\n', best.Rd,
best.Rd*1e3);

        fprintf('  tPW  = %.3f um\n', best.tPW*1e6);
        fprintf('  tUD  = %.3f um\n', best.tUD*1e6);
        fprintf('  thPW = %.3f deg\n', best.thPW);
        fprintf('  thUD = %.3f deg\n\n', best.thUD);

        fprintf('Current best stable states:\n');

        fprintf('  Deployed:\n');
        fprintf('    C = %.4f 1/m\n', best.states.dep.C);
        fprintf('    T = %.4f deg\n', best.states.dep.T);
        fprintf('    |C error| = %.4f 1/m\n',
best.states.dep.errC_abs);
        fprintf('    |T error| = %.4f deg\n',
best.states.dep.errT_abs);
        fprintf('    C error%% = %.4f %%\n',
best.states.dep.errC_pct);

        fprintf('  Coiled:\n');
        fprintf('    C = %.4f 1/m\n', best.states.coil.C);
        fprintf('    T = %.4f deg\n', best.states.coil.T);
        fprintf('    |C error| = %.4f 1/m\n',
best.states.coil.errC_abs);
        fprintf('    |T error| = %.4f deg\n',
best.states.coil.errT_abs);
        fprintf('    C error%% = %.4f %%\n',
best.states.coil.errC_pct);
        fprintf('    T error%% = %.4f %%\n',
best.states.coil.errT_pct);

        fprintf('=====\n');
        end

        if mod(checkCount, progressEvery) == 0 || checkedCount ==
totalComb
            fprintf('Progress: %d / %.0f  (%.1f%%)\n', ...
checkedCount, totalComb, 100*checkedCount/totalComb);
        end
    end
end
end
end

elapsedTime = toc;

fprintf('\nFinished.\n');
fprintf('Checked combinations = %d / %.0f\n', checkedCount, totalComb);
fprintf('Elapsed time = %.2f s (%.2f min)\n\n', elapsedTime, elapsedTime/60);

if ~best.found
    error('No valid parameter set produced two assignable stable states.');
```

end

```

%% =====
% PRINT BEST RESULT
```

```

% =====
fprintf('===== BEST PARAMETER SET =====\n');
fprintf('Rd    = %.6f m    (nominal %.6f m, delta = %+8.6f m)\n', ...
    best.Rd, Rd0, best.Rd - Rd0);

fprintf('tPW   = %.3f um    (nominal %.3f um, delta = %+7.3f um)\n', ...
    best.tPW*1e6, tPW0*1e6, (best.tPW - tPW0)*1e6);

fprintf('tUD   = %.3f um    (nominal %.3f um, delta = %+7.3f um)\n', ...
    best.tUD*1e6, tUD0*1e6, (best.tUD - tUD0)*1e6);

fprintf('thPW  = %.3f deg    (nominal %.3f deg, delta = %+7.3f deg)\n', ...
    best.thPW, thPW0, best.thPW - thPW0);

fprintf('thUD  = %.3f deg    (nominal %.3f deg, delta = %+7.3f deg)\n', ...
    best.thUD, thUD0, best.thUD - thUD0);

fprintf('Objective = %.6f\n', best.obj);
fprintf('===== \n\n');

%% =====
% RESULT TABLE
% =====
fprintf('===== BEST FIT VS EXP =====\n');
fprintf('%-10s %-8s %-12s %-12s %-12s %-12s\n', ...
    'State', 'Var', 'Analytical', 'Experimental', 'Abs Error', 'Error (%)');

fprintf('%-10s %-8s %-12.4f %-12.4f %-12.4f %-12.4f\n', ...
    'Deployed', 'C', ...
    best.states.dep.C, target.C_dep, ...
    best.states.dep.errC_abs, best.states.dep.errC_pct);

fprintf('%-10s %-8s %-12.4f %-12.4f %-12.4f %-12s\n', ...
    'Deployed', 'T', ...
    best.states.dep.T, target.T_dep, ...
    best.states.dep.errT_abs, '-');

fprintf('%-10s %-8s %-12.4f %-12.4f %-12.4f %-12.4f\n', ...
    'Coiled', 'C', ...
    best.states.coil.C, target.C_coil, ...
    best.states.coil.errC_abs, best.states.coil.errC_pct);

fprintf('%-10s %-8s %-12.4f %-12.4f %-12.4f %-12.4f\n', ...
    'Coiled', 'T', ...
    best.states.coil.T, target.T_coil, ...
    best.states.coil.errT_abs, best.states.coil.errT_pct);

fprintf('===== \n');
fprintf('Final counter: %d / %.0f combinations checked\n\n', checkedCount,
totalComb);

%% =====
% PLOT BEST ENERGY SURFACE
% =====
X = Cg .* cosd(2*Tg);
Y = Cg .* sind(2*Tg);

figure('Color','w');
contourf(X, Y, best.U_vals, 50, 'LineColor', 'none');

```

```

colorbar;
xlabel('C cos(2T)');
ylabel('C sin(2T)');
title('Best-fit contour plot of U(T,C)');
axis equal;
grid on;
hold on;

plot(best.states.dep.C*cosd(2*best.states.dep.T), ...
     best.states.dep.C*sind(2*best.states.dep.T), ...
     'rp', 'MarkerSize', 14, 'MarkerFaceColor', 'r', 'LineWidth', 1.5);

plot(best.states.coil.C*cosd(2*best.states.coil.T), ...
     best.states.coil.C*sind(2*best.states.coil.T), ...
     'rp', 'MarkerSize', 14, 'MarkerFaceColor', 'r', 'LineWidth', 1.5);

text(best.states.dep.C*cosd(2*best.states.dep.T), ...
     best.states.dep.C*sind(2*best.states.dep.T), ...
     sprintf(' Deployed\\n C = %.3f\\n T = %.3f^\\circ', ...
     best.states.dep.C, best.states.dep.T), ...
     'Color', 'w', 'FontWeight', 'bold', ...
     'BackgroundColor', 'k', 'Margin', 5);

text(best.states.coil.C*cosd(2*best.states.coil.T), ...
     best.states.coil.C*sind(2*best.states.coil.T), ...
     sprintf(' Coiled\\n C = %.3f\\n T = %.3f^\\circ', ...
     best.states.coil.C, best.states.coil.T), ...
     'Color', 'w', 'FontWeight', 'bold', ...
     'BackgroundColor', 'k', 'Margin', 5);

annotation('textbox', [0.22, 0.01, 0.56, 0.045], ...
     'String', sprintf('Checked combinations: %d / %.0f | Runtime: %.2f min',
     ...
     checkedCount, totalComb, elapsedTime/60), ...
     'EdgeColor', 'none', ...
     'HorizontalAlignment', 'center', ...
     'FontWeight', 'bold', ...
     'FontSize', 10);

%% =====
% LOCAL FUNCTIONS
% =====
function vals = centeredSweep(center, maxDev, step)
% Returns values ordered as:
% center, center+step, center-step, center+2step, center-2step, ...
    vals = center;
    deltas = step:step:maxDev;

    for k = 1:numel(deltas)
        vals = [vals, center + deltas(k), center - deltas(k)]; %#ok<AGROW>
    end

    vals = unique(vals, 'stable');
end

function Qbar = buildQbar(mat, th)
    E11 = mat.E11;
    E22 = mat.E22;
    nu12 = mat.nu12;

```

```

G12 = mat.G12;

den = E11 - nu12^2 * E22;
Qr11 = E11^2 / den;
Qr12 = (nu12 * E11 * E22) / den;
Qr22 = (E11 * E22) / den;
Qr66 = G12;

c = cosd(th);
s = sind(th);

Q11 = Qr11*c^4 + 2*(Qr12 + 2*Qr66)*c^2*s^2 + Qr22*s^4;
Q12 = Qr12*(c^4 + s^4) + (Qr11 + Qr22 - 4*Qr66)*c^2*s^2;
Q16 = (Qr11 - Qr12 - 2*Qr66)*c^3*s - (Qr22 - Qr12 - 2*Qr66)*c*s^3;
Q22 = Qr11*s^4 + 2*(Qr12 + 2*Qr66)*c^2*s^2 + Qr22*c^4;
Q26 = (Qr11 - Qr12 - 2*Qr66)*c*s^3 - (Qr22 - Qr12 - 2*Qr66)*c^3*s;
Q66 = (Qr11 + Qr22 - 2*Qr12 - 2*Qr66)*c^2*s^2 + Qr66*(c^4 + s^4);

Qbar = [Q11, Q12, Q16;
        Q12, Q22, Q26;
        Q16, Q26, Q66];
end

function [A, B, D, DAdj] = buildABD(Qbot, Qud, Qtop, tPW, tUD)
    tTot = tPW + tUD + tPW;

    Z0 = -tTot/2;
    Z1 = Z0 + tPW;
    Z2 = Z1 + tUD;
    Z3 = Z2 + tPW;

    A = Qbot*(Z1-Z0) + Qud*(Z2-Z1) + Qtop*(Z3-Z2);
    B = 0.5*(Qbot*(Z1^2-Z0^2) + Qud*(Z2^2-Z1^2) + Qtop*(Z3^2-Z2^2));
    D = (1/3)*(Qbot*(Z1^3-Z0^3) + Qud*(Z2^3-Z1^3) + Qtop*(Z3^3-Z2^3));

    DAdj = D - (B' * (A\B));
end

function U = energySurface(DAdj, Rd, Cg, Tg)
    v1 = 0.5 * Cg .* (1 - cosd(2*Tg));
    v2 = 0.5 * Cg .* (cosd(2*Tg) + 1 - 2./(Cg.*Rd));
    v3 = 0.5 * Cg .* (2*sind(2*Tg));

    U = 0.5 * ( ...
        DAdj(1,1)*v1.^2 + ...
        2*DAdj(1,2)*v1.*v2 + ...
        2*DAdj(1,3)*v1.*v3 + ...
        DAdj(2,2)*v2.^2 + ...
        2*DAdj(2,3)*v2.*v3 + ...
        DAdj(3,3)*v3.^2 );
end

function minima = extractLocalMinima(U, C_vals, T_vals)
% Local minima with periodic wrapping in T and non-periodic handling in C

    [~, nT] = size(U);
    infRow = inf(1, nT);

    U_up = [infRow; U(1:end-1,:)];

```

```

U_down      = [U(2:end,:); infRow];
U_left      = U(:, [end, 1:end-1]);
U_right     = U(:, [2:end, 1]);
U_up_left   = [infRow; U(1:end-1, [end, 1:end-1])];
U_up_right  = [infRow; U(1:end-1, [2:end, 1])];
U_dn_left   = [U(2:end, [end, 1:end-1]); infRow];
U_dn_right  = [U(2:end, [2:end, 1]); infRow];

isLocalMin = isfinite(U) & ...
              (U <= U_up)      & (U <= U_down)      & ...
              (U <= U_left)   & (U <= U_right)    & ...
              (U <= U_up_left) & (U <= U_up_right) & ...
              (U <= U_dn_left) & (U <= U_dn_right);

[rowMin, colMin] = find(isLocalMin);

if isempty(rowMin)
    minima.C = [];
    minima.T = [];
    minima.U = [];
    return
end

candC = C_vals(rowMin).';
candT = T_vals(colMin).';
candU = U(sub2ind(size(U), rowMin, colMin));

[candU, order] = sort(candU);
candC = candC(order);
candT = candT(order);

tolC = 3 * (C_vals(2) - C_vals(1));
tolT = 3 * (T_vals(2) - T_vals(1));

selC = [];
selT = [];
selU = [];

for k = 1:numel(candU)
    if isempty(selC)
        selC(end+1,1) = candC(k); %#ok<AGROW>
        selT(end+1,1) = candT(k); %#ok<AGROW>
        selU(end+1,1) = candU(k); %#ok<AGROW>
    else
        dC = (candC(k) - selC) / tolC;
        dT = periodicDiff180(candT(k), selT) / tolT;
        dist = sqrt(dC.^2 + dT.^2);

        if all(dist > 1)
            selC(end+1,1) = candC(k); %#ok<AGROW>
            selT(end+1,1) = candT(k); %#ok<AGROW>
            selU(end+1,1) = candU(k); %#ok<AGROW>
        end
    end
end

minima.C = selC;
minima.T = selT;
minima.U = selU;

```

```

end

function [states, ok] = assignStableStates(minima, target)
    ok = false;
    states = struct();

    if numel(minima.C) < 2
        return
    end

    dT_dep = periodicDiff180(minima.T, target.T_dep);
    dT_coil = periodicDiff180(minima.T, target.T_coil);

    depPool = find(dT_dep <= 45);
    coilPool = find(dT_coil <= 45);

    if isempty(depPool) || isempty(coilPool)
        return
    end

    scoreDep = abs(minima.C(depPool) - target.C_dep) / target.C_dep +
dT_dep(depPool) / 90;
    scoreCoil = abs(minima.C(coilPool) - target.C_coil) / target.C_coil +
dT_coil(coilPool) / 90;

    [~, ordDep] = sort(scoreDep);
    [~, ordCoil] = sort(scoreCoil);

    depCandidates = depPool(ordDep);
    coilCandidates = coilPool(ordCoil);

    idxDep = depCandidates(1);
    idxCoil = [];

    for k = 1:numel(coilCandidates)
        if coilCandidates(k) ~= idxDep
            idxCoil = coilCandidates(k);
            break
        end
    end

    if isempty(idxCoil)
        return
    end

    states.dep.C = minima.C(idxDep);
    states.dep.T = minima.T(idxDep);
    states.dep.U = minima.U(idxDep);

    states.coil.C = minima.C(idxCoil);
    states.coil.T = minima.T(idxCoil);
    states.coil.U = minima.U(idxCoil);

    states.dep.errC_abs = abs(states.dep.C - target.C_dep);
    states.dep.errT_abs = periodicDiff180(states.dep.T, target.T_dep);
    states.coil.errC_abs = abs(states.coil.C - target.C_coil);
    states.coil.errT_abs = periodicDiff180(states.coil.T, target.T_coil);

    states.dep.errC_pct = 100 * states.dep.errC_abs / target.C_dep;

```

```

        states.coil.errC_pct = 100 * states.coil.errC_abs / target.C_coil;
        states.coil.errT_pct = 100 * states.coil.errT_abs / target.T_coil;

        ok = true;
    end

function d = periodicDiff180(a, b)
    d = abs(mod((a - b) + 90, 180) - 90);
end

% Optimization Approach
clear
clc
close all

%% =====
% DESIGN GOAL
% Minimize the energy barrier between deployed-like and
% coiled-like states for symmetric 3-ply laminates [outer/mid/outer]
%
% Candidate layups:
% 1) UD / UD / UD
% 2) UD / PW / UD
% 3) PW / UD / PW
% 4) PW / PW / PW
%
% Outer and middle ply angles are varied independently.
% Top and bottom plies share the same angle to maintain symmetry.
%% =====

%% =====
% MATERIAL PROPERTIES
% =====
matUD.E11 = 144.1e9;
matUD.E22 = 5.2e9;
matUD.nu12 = 0.335;
matUD.G12 = 2.8e9;

matPW.E11 = 89e9;
matPW.E22 = 89e9;
matPW.nu12 = 0.035;
matPW.G12 = 4.2e9;

%% =====
% NOMINAL GEOMETRY / THICKNESS
% Keep geometry close to original by staying with 3 plies only
%% =====
Rd = 0.5 * 0.060325; % m

tPW0 = 58.2e-6; % m
tUD0 = 40e-6; % m

%% =====
% ANGLE SEARCH SETTINGS
% Use large increments as requested
%% =====
dAng = 5; % deg, change to 10 if desired
thetaOuter_vals = -90:dAng:90; % outer plies

```

```

thetaMid_vals    = -90:dAng:90;      % middle ply

%% =====
% T-SEARCH SETTINGS
% Faster 1D-in-T search with analytic optimal C(T)
%% =====
T_vals = linspace(0, 180, 721);      % 0.25 deg resolution
T_vals(end) = [];                    % remove 180 because it is equivalent to 0

Cmin = 1e-6;
Cmax = 90;

% Windows used to identify the two stable states
deployWindow = 25;    % deg around T = 0
coilWindow    = 25;    % deg around T = 90

% Barrier path sampling
nPath = 300;

%% =====
% CANDIDATE SYMMETRIC 3-PLY LAYUPS
%% =====
layups = struct([]);

layups(1).name      = 'UD / UD / UD';
layups(1).materials = {'UD','UD','UD'};
layups(1).thickness = [tUD0, tUD0, tUD0];

layups(2).name      = 'UD / PW / UD';
layups(2).materials = {'UD','PW','UD'};
layups(2).thickness = [tUD0, tPW0, tUD0];

layups(3).name      = 'PW / UD / PW';
layups(3).materials = {'PW','UD','PW'};
layups(3).thickness = [tPW0, tUD0, tPW0];

layups(4).name      = 'PW / PW / PW';
layups(4).materials = {'PW','PW','PW'};
layups(4).thickness = [tPW0, tPW0, tPW0];

%% =====
% PRINT SEARCH SIZE
%% =====
nLay    = numel(layups);
nOuter  = numel(thetaOuter_vals);
nMid    = numel(thetaMid_vals);

totalComb = nLay * nOuter * nMid;

fprintf('\n===== OPTIMIZED CONFIGURATION SEARCH =====\n');
fprintf('Symmetric layups tested   : %d\n', nLay);
fprintf('Outer angle count           : %d\n', nOuter);
fprintf('Middle angle count          : %d\n', nMid);
fprintf('Total combinations          : %d\n', totalComb);
fprintf('Angle increment             : %.1f deg\n', dAng);
fprintf('Reference radius            : %.6f m\n', Rd);
fprintf('===== \n\n');

%% =====

```

```

% EXHAUSTIVE SEARCH
% Best = minimum common barrier:
% barrierCommon = saddleEnergy - max(U_dep, U_coil)
%
% Also report:
% barrierFromDep = saddleEnergy - U_dep
% barrierFromCoil = saddleEnergy - U_coil
%% =====
best.found = false;
best.barrierCommon = inf;

checkedCount = 0;
progressEvery = max(1, round(totalComb / 50)); % every ~2%

tic

for iLay = 1:nLay
    currentLayup = layups(iLay);

    for iOuter = 1:nOuter
        thetaOuter = thetaOuter_vals(iOuter);

        for iMid = 1:nMid
            thetaMid = thetaMid_vals(iMid);

            checkedCount = checkedCount + 1;

            % Build Qbar for each ply
            Qbars = cell(1,3);
            thetas = [thetaOuter, thetaMid, thetaOuter];

            for k = 1:3
                matName = currentLayup.materials{k};
                if strcmp(matName, 'UD')
                    mat = matUD;
                else
                    mat = matPW;
                end
                Qbars{k} = buildQbar(mat, thetas(k));
            end

            % ABD
            [A, B, D, DAdj] = buildABD_general(Qbars, currentLayup.thickness);

            if rcond(A) < 1e-14 || any(~isfinite(DAdj(:)))
                continue
            end

            % Fast search over T only
            [Copt, Uopt] = optimizeCoverT(DAdj, Rd, T_vals, Cmin, Cmax);

            % Extract T-minima from U(T)
            minima = extract1DMinima(T_vals, Copt, Uopt);

            if numel(minima.T) < 2
                continue
            end

            % Identify deployed-like and coiled-like minima

```

```

[states, ok] = assignByStateWindows(minima, deployWindow, coilWindow);

if ~ok
    continue
end

% Approximate transition barrier along straight path in (C,T)
[barrierCommon, barrierFromDep, barrierFromCoil, saddleU] = ...
    transitionBarrier(DAdj, Rd, ...
        states.dep.C, states.dep.T, states.dep.U, ...
        states.coil.C, states.coil.T, states.coil.U, ...
        nPath);

if barrierCommon < best.barrierCommon
    best.found = true;

    best.barrierCommon = barrierCommon;
    best.barrierFromDep = barrierFromDep;
    best.barrierFromCoil = barrierFromCoil;
    best.saddleU = saddleU;

    best.layupName = currentLayup.name;
    best.materials = currentLayup.materials;
    best.thickness = currentLayup.thickness;
    best.thetaOuter = thetaOuter;
    best.thetaMid = thetaMid;

    best.states = states;
    best.Copt = Copt;
    best.Uopt = Uopt;

    fprintf('\n===== \n');
    fprintf('NEW BEST CONFIGURATION FOUND \n');
    fprintf('Iteration      : %d / %d (%.2f%%) \n', ...
        checkedCount, totalComb, 100*checkedCount/totalComb);
    fprintf('Layup              : %s \n', best.layupName);
    fprintf('theta_outer        : %.1f deg \n', best.thetaOuter);
    fprintf('theta_middle       : %.1f deg \n', best.thetaMid);
    fprintf('Barrier (common)   : %.6e \n', best.barrierCommon);
    fprintf('Barrier from dep   : %.6e \n', best.barrierFromDep);
    fprintf('Barrier from coil  : %.6e \n', best.barrierFromCoil);

    fprintf('\nStable states: \n');
    fprintf('  Deployed-like: C = %.4f 1/m, T = %.4f deg, U = %.6e \n',
...
        best.states.dep.C, best.states.dep.T, best.states.dep.U);
    fprintf('  Coiled-like   : C = %.4f 1/m, T = %.4f deg, U = %.6e \n',
...
        best.states.coil.C, best.states.coil.T, best.states.coil.U);
    fprintf('===== \n');
end

if mod(checkedCount, progressEvery) == 0 || checkedCount == totalComb
    fprintf('Progress: %d / %d (%.1f%%) \n', ...
        checkedCount, totalComb, 100*checkedCount/totalComb);
end
end
end
end
end

```

```

elapsedTime = toc;

fprintf('\nFinished.\n');
fprintf('Checked combinations = %d / %d\n', checkedCount, totalComb);
fprintf('Elapsed time = %.2f s (%.2f min)\n\n', elapsedTime, elapsedTime/60);

if ~best.found
    error('No valid bistable configuration found in the prescribed search space.');
```

```

end

%% =====
% FINAL BEST RESULT
%% =====
fprintf('===== BEST OPTIMIZED CONFIGURATION =====\n');
fprintf('Layup           : %s\n', best.layupName);
fprintf('Outer ply angle    : %.1f deg\n', best.thetaOuter);
fprintf('Middle ply angle    : %.1f deg\n', best.thetaMid);
fprintf('Barrier (common)    : %.6e\n', best.barrierCommon);
fprintf('Barrier from dep    : %.6e\n', best.barrierFromDep);
fprintf('Barrier from coil   : %.6e\n', best.barrierFromCoil);
fprintf('Saddle energy       : %.6e\n', best.saddleU);

fprintf('\nStable states:\n');
fprintf('  Deployed-like: C = %.4f 1/m, T = %.4f deg, U = %.6e\n', ...
    best.states.dep.C, best.states.dep.T, best.states.dep.U);
fprintf('  Coiled-like   : C = %.4f 1/m, T = %.4f deg, U = %.6e\n', ...
    best.states.coil.C, best.states.coil.T, best.states.coil.U);
fprintf('===== \n');

%% =====
% PLOT BEST U*(T) CURVE
%% =====
figure('Color','w');
plot(T_vals, best.Uopt, 'LineWidth', 1.5);
grid on;
xlabel('T (deg)');
ylabel('Optimized U(T) with C = C^(T)');
title(sprintf('Best configuration: %s | \theta_o = %.1f^\circ, \theta_m = %.1f^\circ', ...
    best.layupName, best.thetaOuter, best.thetaMid));
hold on;

plot(best.states.dep.T, best.states.dep.U, 'ro', 'MarkerFaceColor', 'r');
plot(best.states.coil.T, best.states.coil.U, 'bo', 'MarkerFaceColor', 'b');

text(best.states.dep.T, best.states.dep.U, ...
    sprintf('  Dep: C=%.2f, T=%.2f^\circ', best.states.dep.C, best.states.dep.T),
    ...
    'VerticalAlignment','bottom');

text(best.states.coil.T, best.states.coil.U, ...
    sprintf('  Coil: C=%.2f, T=%.2f^\circ', best.states.coil.C,
    best.states.coil.T), ...
    'VerticalAlignment','top');

%% =====
% LOCAL FUNCTIONS

```

```

%% =====
function Qbar = buildQbar(mat, th)
    E11 = mat.E11;
    E22 = mat.E22;
    nu12 = mat.nu12;
    G12 = mat.G12;

    den = E11 - nu12^2 * E22;
    Qr11 = E11^2 / den;
    Qr12 = (nu12 * E11 * E22) / den;
    Qr22 = (E11 * E22) / den;
    Qr66 = G12;

    c = cosd(th);
    s = sind(th);

    Q11 = Qr11*c^4 + 2*(Qr12 + 2*Qr66)*c^2*s^2 + Qr22*s^4;
    Q12 = Qr12*(c^4 + s^4) + (Qr11 + Qr22 - 4*Qr66)*c^2*s^2;
    Q16 = (Qr11 - Qr12 - 2*Qr66)*c^3*s - (Qr22 - Qr12 - 2*Qr66)*c*s^3;
    Q22 = Qr11*s^4 + 2*(Qr12 + 2*Qr66)*c^2*s^2 + Qr22*c^4;
    Q26 = (Qr11 - Qr12 - 2*Qr66)*c*s^3 - (Qr22 - Qr12 - 2*Qr66)*c^3*s;
    Q66 = (Qr11 + Qr22 - 2*Qr12 - 2*Qr66)*c^2*s^2 + Qr66*(c^4 + s^4);

    Qbar = [Q11, Q12, Q16;
            Q12, Q22, Q26;
            Q16, Q26, Q66];
end

function [A, B, D, DAdj] = buildABD_general(Qbars, thicknesses)
    n = numel(thicknesses);
    z = zeros(1, n+1);
    z(1) = -sum(thicknesses)/2;

    for k = 1:n
        z(k+1) = z(k) + thicknesses(k);
    end

    A = zeros(3);
    B = zeros(3);
    D = zeros(3);

    for k = 1:n
        Qk = Qbars{k};
        zk0 = z(k);
        zk1 = z(k+1);

        A = A + Qk*(zk1-zk0);
        B = B + 0.5*Qk*(zk1^2-zk0^2);
        D = D + (1/3)*Qk*(zk1^3-zk0^3);
    end

    DAdj = D - (B' * (A\B));
end

function [Copt, Uopt] = optimizeCoverT(DAdj, Rd, T_vals, Cmin, Cmax)
    c2 = cosd(2*T_vals);
    s2 = sind(2*T_vals);

    p1 = 0.5*(1 - c2);

```

```

p2 = 0.5*(1 + c2);
p3 = s2;

P = [p1; p2; p3];           % 3 x N
DP = DAdj * P;              % 3 x N

a = sum(P .* DP, 1);        % 1 x N

q = [0; -1/Rd; 0];
Dq = DAdj * q;
b = sum(P .* Dq, 1);        % 1 x N
c0 = q' * DAdj * q;

Copt = -b ./ a;

invalid = (a <= 0) | ~isfinite(Copt);
Copt(invalid) = NaN;

Copt = max(Cmin, min(Cmax, Copt));

Uopt = 0.5 * (a .* Copt.^2 + 2*b .* Copt + c0);
Uopt(invalid) = inf;
end

function minima = extract1DMinima(T_vals, Copt, Uopt)
    isMin = isfinite(Uopt) & ...
        (Uopt <= circshift(Uopt, 1)) & ...
        (Uopt <= circshift(Uopt, -1));

    idx = find(isMin);

    if isempty(idx)
        minima.T = [];
        minima.C = [];
        minima.U = [];
        return
    end

    candT = T_vals(idx).';
    candC = Copt(idx).';
    candU = Uopt(idx).';

    [candU, order] = sort(candU);
    candT = candT(order);
    candC = candC(order);

    tolT = 3 * abs(T_vals(2) - T_vals(1));

    selT = [];
    selC = [];
    selU = [];

    for k = 1:numel(candU)
        if isempty(selT)
            selT(end+1,1) = candT(k); %#ok<AGROW>
            selC(end+1,1) = candC(k); %#ok<AGROW>
            selU(end+1,1) = candU(k); %#ok<AGROW>
        else
            dT = periodicDiff180(candT(k), selT);

```

```

        if all(dT > tolT)
            selT(end+1,1) = candT(k); %#ok<AGROW>
            selC(end+1,1) = candC(k); %#ok<AGROW>
            selU(end+1,1) = candU(k); %#ok<AGROW>
        end
    end
end

minima.T = selT;
minima.C = selC;
minima.U = selU;
end

function [states, ok] = assignByStateWindows(minima, deployWindow, coilWindow)
    ok = false;
    states = struct();

    if numel(minima.T) < 2
        return
    end

    dDep = periodicDiff180(minima.T, 0);
    dCoil = periodicDiff180(minima.T, 90);

    depPool = find(dDep <= deployWindow);
    coilPool = find(dCoil <= coilWindow);

    if isempty(depPool) || isempty(coilPool)
        return
    end

    [~, i1] = min(minima.U(depPool));
    [~, i2] = min(minima.U(coilPool));

    idxDep = depPool(i1);
    idxCoil = coilPool(i2);

    if idxDep == idxCoil
        return
    end

    states.dep.T = minima.T(idxDep);
    states.dep.C = minima.C(idxDep);
    states.dep.U = minima.U(idxDep);

    states.coil.T = minima.T(idxCoil);
    states.coil.C = minima.C(idxCoil);
    states.coil.U = minima.U(idxCoil);

    ok = true;
end

function [barrierCommon, barrierFromDep, barrierFromCoil, saddleU] = ...
    transitionBarrier(DAdj, Rd, C1, T1, U1, C2, T2, U2, nPath)

    lambda = linspace(0, 1, nPath);
    Cpath = (1-lambda)*C1 + lambda*C2;
    Tpath = (1-lambda)*T1 + lambda*T2;

```

```

    Upath = energyScalar(DAdj, Rd, Cpath, Tpath);
    saddleU = max(Upath);

    barrierFromDep = max(0, saddleU - U1);
    barrierFromCoil = max(0, saddleU - U2);
    barrierCommon = max(0, saddleU - max(U1, U2));
end

function U = energyScalar(DAdj, Rd, C, T)
    v1 = 0.5 * C .* (1 - cosd(2*T));
    v2 = 0.5 * C .* (cosd(2*T) + 1 - 2./(C.*Rd));
    v3 = 0.5 * C .* (2*sind(2*T));

    U = 0.5 * ( ...
        DAdj(1,1)*v1.^2 + ...
        2*DAdj(1,2)*v1.*v2 + ...
        2*DAdj(1,3)*v1.*v3 + ...
        DAdj(2,2)*v2.^2 + ...
        2*DAdj(2,3)*v2.*v3 + ...
        DAdj(3,3)*v3.^2 );
end

function d = periodicDiff180(a, b)
    d = abs(mod((a - b) + 90, 180) - 90);
end

```